

02 · Tables and DDL

Why this matters in interviews. "*DELETE vs TRUNCATE vs DROP*" is one of the most-asked SQL questions of all. Interviewers also check that you can `ALTER` a table without looking it up, and know how Oracle generates ids (no `AUTO_INCREMENT`).

In this chapter: `CREATE TABLE` · copying tables · `ALTER TABLE` · `DROP`, `TRUNCATE`, `DELETE` compared · `RENAME` · auto-generated keys (identity, sequences, `AUTO_INCREMENT`) · column defaults · looking at a table's structure.

Constraints (`PRIMARY KEY`, `FOREIGN KEY`, `CHECK` ...) are so central that they have their own chapter — **03 · Constraints and Keys**. Here we only touch them where DDL syntax needs it.

2.1 `CREATE TABLE`

A table is created with its **name**, and for every column a **name** and a **data type** (plus optional defaults and constraints).

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    column3 datatype,  
    ....  
);
```

A realistic Oracle example:

```
CREATE TABLE student (  
    student_id  NUMBER(6)      PRIMARY KEY,  
    full_name   VARCHAR2(60)  NOT NULL,  
    email       VARCHAR2(100),  
    joined_on   DATE          DEFAULT SYSDATE,  
    marks       NUMBER(5,2)  
);
```

Naming rules (Oracle). Start with a letter · letters, digits, `_`, `$`, `#` only · at most 30 bytes (128 from 12.2) · not a reserved word · unquoted names are stored in **uppercase**.

Copying a table: `CREATE TABLE ... AS SELECT` (CTAS)

An existing table can be copied with `CREATE TABLE ... AS SELECT`. The new table gets the **same column definitions** and is **filled with the selected rows**. You can choose all columns or only some, and you can filter rows with `WHERE`.

```
CREATE TABLE new_table_name AS
  SELECT column1, column2, ...
  FROM   existing_table_name
  WHERE  ....;
```

```
CREATE TABLE engineers AS
  SELECT emp_id, emp_name, salary
  FROM   emp
  WHERE  dept_id = 20;
```

Table after

emp_id	emp_name	salary
102	Bala	70000
103	Chitra	50000
104	Dinesh	50000
105	Esha	40000
109	Isha	20000

Very useful variations:

Goal	Statement
Copy structure and data	<code>CREATE TABLE emp_backup AS SELECT * FROM emp;</code>
Copy structure only (empty table)	<code>CREATE TABLE emp_empty AS SELECT * FROM emp WHERE 1 = 0;</code>
Copy only some columns/rows	<code>CREATE TABLE emp_names AS SELECT emp_id, emp_name FROM emp WHERE dept_id = 20;</code>
Rename columns on the way	<code>CREATE TABLE e2 AS SELECT emp_id AS id, salary AS pay FROM emp;</code>

What CTAS does *not* copy. Column names, data types and `NOT NULL` come across. **Primary / foreign / unique / check constraints, indexes and triggers do not** — recreate them on the copy if you need them.

Oracle vs others — `SELECT ... INTO`. SQL Server and MS Access copy into a new table with `SELECT * INTO newtable [IN externaldb] FROM oldtable WHERE ...;` (for example `SELECT * INTO CustomersBackup2017 FROM Customers;`). **Oracle does not support this.** In Oracle SQL use CTAS; in **PL/SQL**, `SELECT ... INTO` means something else entirely — *assign the result to variables* (Chapter 12).

2.2 ALTER TABLE

`ALTER TABLE` **adds, deletes or modifies columns** of an existing table, and **adds or drops constraints** (Chapter 03).

Add a column

```
ALTER TABLE table_name ADD column_name datatype;
ALTER TABLE Customers ADD Email varchar(255);
```

Oracle can add several at once — note the parentheses: `ALTER TABLE emp ADD (phone VARCHAR2(15), grade CHAR(1));`

Drop a column

```
ALTER TABLE table_name DROP COLUMN column_name;
ALTER TABLE Customers DROP COLUMN Email;
```

Some database systems do not allow deleting a column. Oracle does: `DROP COLUMN c`, several at once with `DROP (c1, c2)`, and for very large tables you can first `SET UNUSED COLUMN c` (instant) and drop it later.

Rename a column

```
ALTER TABLE table_name RENAME COLUMN old_name TO new_name;
```

(MySQL 5.x uses `CHANGE old_name new_name datatype`; MySQL 8, PostgreSQL and Oracle use `RENAME COLUMN ... TO .`)

Change (modify) a column's data type

The keyword differs by database:

Database	Syntax
SQL Server / MS Access	<code>ALTER TABLE table_name ALTER COLUMN column_name datatype;</code>
MySQL / Oracle (prior to 10g)	<code>ALTER TABLE table_name MODIFY COLUMN column_name datatype;</code>
Oracle 10g and later	<code>ALTER TABLE table_name MODIFY column_name datatype;</code>
PostgreSQL	<code>ALTER TABLE table_name ALTER COLUMN column_name TYPE datatype;</code>

Example (W3Schools): change `DateOfBirth` in `Persons` to type `year` — `ALTER TABLE Persons ALTER COLUMN DateOfBirth year;` — after which the column holds a two- or four-digit year. (`year` is a MySQL type; the pattern is what matters.) Then `ALTER TABLE Persons DROP COLUMN DateOfBirth;` removes the column entirely.

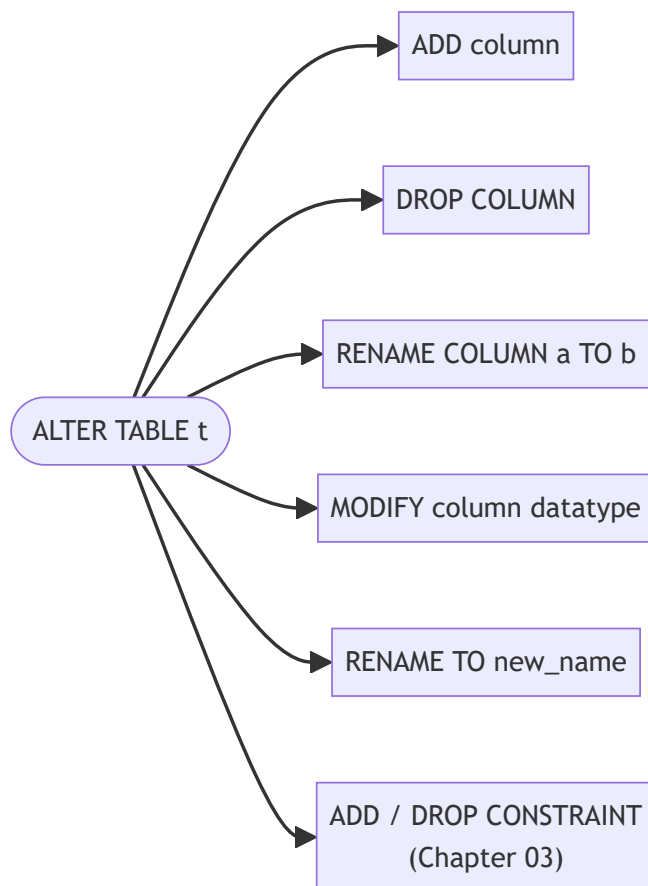
Oracle restrictions when modifying a column:

You want to ...	Allowed when ...
Increase length / precision	always
Decrease length / precision	only if every existing value still fits
Change to a different data type	the column is empty (all NULL) — or the conversion is between compatible types such as <code>CHAR</code> ↔ <code>VARCHAR2</code>
Add <code>NOT NULL</code>	no existing row has NULL there (else <code>ORA-02296</code>)

Rename a table

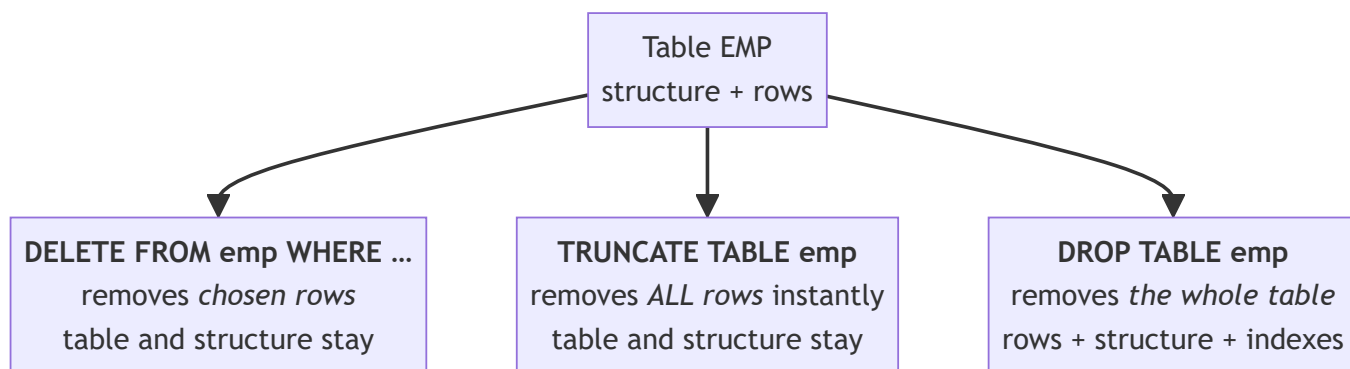
```
RENAME old_table TO new_table;           -- Oracle shortcut
ALTER TABLE old_table RENAME TO new_table; -- also standard for Oracle / PostgreSQL / MySQL
```

Everything `ALTER TABLE` can do — at a glance



2.3 `DROP`, `TRUNCATE`, `DELETE` — the classic comparison

Three ways to make data disappear, each removing a different amount:



	DELETE	TRUNCATE	DROP
Category	DML	DDL	DDL
Removes	chosen rows (WHERE) or all	all rows	data and the table definition
WHERE clause	✓ yes	✗ no	✗ no
Table structure afterwards	stays	stays	gone
Rollback	✓ yes, until COMMIT	✗ no (auto-commit)	✗ no (auto-commit)
Speed	slowest — row by row, writes undo	very fast — deallocates storage	fast
Fires DELETE triggers	✓ yes	✗ no	✗ no
Space / high-water mark	space stays reserved	released , HWM reset	released
Indexes / constraints	kept	kept (data cleared)	dropped with the table
Identity / auto-increment counter	not reset	reset in MySQL and SQL Server; not reset in Oracle (sequences are separate objects)	gone

```

DELETE FROM Customers WHERE Country = 'Mexico';    -- some rows
DELETE FROM Customers;                             -- ALL rows, but rollback still possible
TRUNCATE TABLE Customers;                         -- ALL rows, instantly, no rollback
DROP TABLE Customers;                             -- table itself is gone

```

Trap — the missing WHERE. DELETE FROM emp; and UPDATE emp SET salary = 0; without a WHERE change **every row**. That is why you should COMMIT only after checking SELECT COUNT(*) .

Oracle details worth quoting:

- **DROP TABLE goes to the recycle bin** (10g+). You can undo it: FLASHBACK TABLE emp TO BEFORE DROP; . To bypass it: DROP TABLE emp PURGE; .
- If other tables have foreign keys pointing at it, DROP TABLE dept; fails; use DROP TABLE dept CASCADE CONSTRAINTS; . This is also why the order matters: **drop the child table (employee) before the parent (department)**.
- TRUNCATE on a table referenced by an *enabled* foreign key fails with ORA-02266 , even if the child table is empty — disable the constraint or truncate the child first.
- TRUNCATE needs no WHERE because it doesn't scan rows; it resets the segment's high-water mark.

Oracle vs others. In PostgreSQL and SQL Server, TRUNCATE **can** be rolled back inside a transaction. The blanket statement "TRUNCATE cannot be rolled back" is a statement about Oracle and MySQL.

2.4 Auto-generated keys

A surrogate key — a number the database generates so you don't have to — is usually the primary key.

MySQL: `AUTO_INCREMENT`

```
CREATE TABLE Persons (  
  Personid  int NOT NULL AUTO_INCREMENT,  
  LastName  varchar(255) NOT NULL,  
  FirstName varchar(255),  
  Age       int,  
  PRIMARY KEY (Personid)  
);
```

- The counter **starts at 1** and adds **1** per new row.
- Start elsewhere: `ALTER TABLE Persons AUTO_INCREMENT = 100;`
- You **leave the column out** of the `INSERT` and a unique value is assigned.

```
INSERT INTO Employees (Name) VALUES ('Alice');  
INSERT INTO Employees (Name) VALUES ('Bob');
```

EmpID	Name
1	Alice
2	Bob

Rules and limits

- **Deleting rows does not reset the counter.** Delete everything and the next id continues from the last number used unless you reset it manually.
- **Only one** `AUTO_INCREMENT` column per table, and it **must be a key** (usually the primary key).
- **Gaps happen** — if rows are deleted or an insert fails, the number that was reserved is not reused.

The same idea in other databases

Database	Syntax	Notes
MySQL / SQLite	<code>AUTO_INCREMENT</code> / <code>AUTOINCREMENT</code>	starts 1, step 1
PostgreSQL	<code>SERIAL</code> , or <code>GENERATED ... AS IDENTITY</code>	<code>SERIAL</code> is a shorthand for a sequence + default
SQL Server	<code>IDENTITY(seed, increment)</code>	e.g. <code>IDENTITY(1,1)</code> ; increment can be customised
Oracle 12c+	<code>GENERATED ... AS IDENTITY</code>	see below
Oracle (any version)	<code>SEQUENCE</code>	see below

Oracle 12c+: identity columns

```
CREATE TABLE employees (  
    employee_id    NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    employee_name  VARCHAR2(100)  
);
```

`GENERATED ALWAYS AS IDENTITY` means **the database automatically generates a unique number** for that column when you insert a row. It is similar to MySQL's `AUTO_INCREMENT`, but more flexible and standards-compliant. There are **three variations**:

Clause	Behaviour
<code>GENERATED ALWAYS AS IDENTITY</code>	Oracle always generates the value. You cannot supply one — unless you write <code>OVERRIDING SYSTEM VALUE</code>
<code>GENERATED BY DEFAULT AS IDENTITY</code>	Oracle generates a value if you don't give one ; you <i>can</i> insert your own with no special syntax
<code>GENERATED BY DEFAULT ON NULL AS IDENTITY (12.1+)</code>	Oracle generates a value only if the column value is NULL — handy when you want automatic <i>and</i> manual id entry

```
-- rejected by GENERATED ALWAYS:  
INSERT INTO employees (employee_id, employee_name) VALUES (100, 'John Doe');  
  
-- accepted, because you explicitly override:  
INSERT INTO employees (employee_id, employee_name)  
OVERRIDING SYSTEM VALUE  
VALUES (100, 'John Doe');
```

You can tune the generator: `GENERATED ALWAYS AS IDENTITY (START WITH 100 INCREMENT BY 10)`.

Oracle (all versions): sequences

Before 12c — and still very common — Oracle uses a separate **sequence object** that is not tied to any table:

```
CREATE SEQUENCE emp_seq START WITH 1 INCREMENT BY 1 CACHE 20;  
  
INSERT INTO emp (emp_id, emp_name) VALUES (emp_seq.NEXTVAL, 'Kavya');    -- next number  
SELECT emp_seq.CURRVAL FROM dual;                                         -- last number *this  
session* got
```

`NEXTVAL` gives a new number; `CURRVAL` returns the one your session last received (valid only *after* a `NEXTVAL` in the same session). Because a sequence is independent of the table:

- numbers are **never returned** by `ROLLBACK`, so **gaps are normal**;
- `CACHE` pre-allocates numbers in memory, so an instance restart can skip a block;
- one sequence can feed several tables.

Pre-identity Oracle code often filled the id with a trigger: `BEFORE INSERT ... FOR EACH ROW BEGIN :NEW.emp_id := emp_seq.NEXTVAL; END;` (Chapter 15).

2.5 Column defaults

`DEFAULT` supplies a value when an `INSERT` does not mention the column.

```
CREATE TABLE Users (  
    UserID INT PRIMARY KEY,  
    Country VARCHAR(50) DEFAULT 'India'  
);  
-- if Country is not given, 'India' is inserted automatically
```

Oracle notes:

- Put `DEFAULT` **before** any inline constraint: `location VARCHAR2(50) DEFAULT 'Unknown' NOT NULL`.
- `DEFAULT` can be an expression such as `SYSDATE`, e.g. `hire_date DATE DEFAULT SYSDATE`.
- A default fires **only when the column is omitted** — an explicit `NULL` is stored as `NULL`. (`DEFAULT ON NULL` changes that.)
- `DEFAULT` is not a constraint. It cannot be given a `CONSTRAINT name` (Chapter 03).

2.6 Inspecting what exists (Oracle data dictionary)

```
DESC emp;                                -- columns, types, nullability (SQL*Plus / SQL  
Developer)  
  
SELECT table_name FROM user_tables;      -- tables you own  
SELECT column_name, data_type, data_length, nullable  
FROM user_tab_columns WHERE table_name = 'EMP'; -- note: stored in UPPERCASE  
SELECT constraint_name, constraint_type  
FROM user_constraints WHERE table_name = 'EMP'; -- P=primary, R=foreign, U=unique, C=check/not  
null
```

The `USER_*` views show your objects; `ALL_*` show what you can see; `DBA_*` show everything (DBA only).

Interview corner

Q. Difference between `DELETE`, `TRUNCATE` and `DROP`? `DELETE` (DML) removes rows one by one, supports `WHERE`, fires triggers, can be rolled back. `TRUNCATE` (DDL) removes all rows instantly, no `WHERE`, auto-commits, keeps the table. `DROP` (DDL) removes the whole table — data, structure, indexes.

Q. Which is faster, `DELETE` or `TRUNCATE`? Why? `TRUNCATE` — it deallocates storage and resets the high-water mark instead of logging every row.

Q. Can you roll back a `TRUNCATE`? In Oracle and MySQL, no (it is DDL and auto-commits). In PostgreSQL and SQL Server, yes inside a transaction.

Q. How do you copy a table's structure without data? `CREATE TABLE copy AS SELECT * FROM src WHERE 1 = 0;`

Q. How do you add an auto-incrementing id in Oracle? 12c+: `GENERATED ALWAYS AS IDENTITY` (or `BY DEFAULT`). Earlier: a `SEQUENCE` with `NEXTVAL`, optionally in a `BEFORE INSERT` trigger.

Q. Why are there gaps in my identity/sequence numbers? Numbers are reserved before the insert commits; `ROLLBACK`, failed inserts, deleted rows and `CACHE` loss are never given back.

Q. Change a column's data type in Oracle? `ALTER TABLE t MODIFY col new_type;` — allowed for a different type only if the column is empty; sizes can grow always and shrink only if data fits.

Q. What happens to the table if you `DROP` it by mistake in Oracle? It goes to the recycle bin — `FLASHBACK TABLE t TO BEFORE DROP;` restores it (unless dropped with `PURGE`).

Quick recap

- `CREATE TABLE` names columns + types; **CTAS** copies data, and `WHERE 1 = 0` copies only the structure. CTAS does **not** copy keys, indexes or triggers.
- `ALTER TABLE`: `ADD`, `DROP COLUMN`, `RENAME COLUMN a TO b`, `MODIFY` (Oracle 10g+) / `ALTER COLUMN` (SQL Server) / `MODIFY COLUMN` (MySQL).
- `DELETE` = DML, rollback-able, `WHERE`; `TRUNCATE` = DDL, all rows, fast, no rollback; `DROP` = whole table (recycle bin in Oracle).
- Ids: MySQL `AUTO_INCREMENT` · SQL Server `IDENTITY` · PostgreSQL `SERIAL` · Oracle **IDENTITY** (12c+) or **SEQUENCE**. Deleting rows never resets the counter; gaps are normal.
- `DEFAULT` applies only when a column is omitted, and is not a named constraint.